

Advanced SQL

Name: DURGASREE AVVARU

Registered Mail: 2300031591cseh@gmail.com

Exercise 1: Control Structures:

Scenario 1:

```
CREATE TABLE Customers (  
    CustomerID NUMBER PRIMARY KEY,  
    CustomerName VARCHAR2(50),  
    Age NUMBER  
);  
  
CREATE TABLE Loans (  
    LoanID NUMBER PRIMARY KEY,  
    CustomerID NUMBER,  
    LoanAmount NUMBER,  
    InterestRate NUMBER,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);  
  
INSERT INTO Customers VALUES (1, 'Praneeth', 65);  
INSERT INTO Customers VALUES (2, 'Rahul', 45);  
INSERT INTO Customers VALUES (3, 'Arjun', 70);  
INSERT INTO Customers VALUES (4, 'Kiran', 55);  
INSERT INTO Loans VALUES (101, 1, 500000, 10);  
INSERT INTO Loans VALUES (102, 2, 300000, 9);  
INSERT INTO Loans VALUES (103, 3, 700000, 11);  
INSERT INTO Loans VALUES (104, 4, 400000, 8);  
SELECT * FROM Customers;  
SELECT * from Loans;  
UPDATE Loans  
SET InterestRate = InterestRate - 1  
WHERE CustomerID IN (  
    SELECT CustomerID
```

```

FROM Customers
WHERE Age > 60
);
SELECT * FROM Loans;

```

Output Screenshot:

The screenshot shows a SQL IDE interface with a script editor and a results pane. The script defines a `Loans` table with columns `LoanAmount` and `InterestRate`, and a foreign key to the `Customers` table. It then inserts data into both tables and updates the interest rate for loans of customers aged 60 and above. The results pane shows the output of the `SELECT * FROM Loans;` statement, which returned 4 rows.

```

10      LoanAmount NUMBER,
11      InterestRate NUMBER,
12      FOREIGN KEY (CustomerID) REFERENCES Customers (CustomerID)
13  );
14
15  INSERT INTO Customers VALUES (1, 'Praneeth', 65);
16  INSERT INTO Customers VALUES (2, 'Rahul', 45);
17  INSERT INTO Customers VALUES (3, 'Arjun', 70);
18  INSERT INTO Customers VALUES (4, 'Kiran', 55);
19
20  INSERT INTO Loans VALUES (101, 1, 500000, 10);
21  INSERT INTO Loans VALUES (102, 2, 300000, 9);
22  INSERT INTO Loans VALUES (103, 3, 700000, 11);
23  INSERT INTO Loans VALUES (104, 4, 400000, 8);
24
25  SELECT * FROM Customers;
26  SELECT * from Loans;=====
27
28  UPDATE Loans
29  SET InterestRate = InterestRate - 1
30  WHERE CustomerID IN (
31      SELECT CustomerID
32      FROM Customers
33      WHERE Age > 60
34  );

```

	LoanID	CustomerID	LoanAmount	InterestRate
1	101	1	500000	9
2	102	2	300000	9
3	103	3	700000	10
4	104	4	400000	8

Execution finished without errors.
 Result: 4 rows returned in 10ms
 At line 36:
 SELECT * FROM Loans;

Scenario 2:

```
CREATE TABLE Customers1 (  
    CustomerID INTEGER PRIMARY KEY,  
    CustomerName TEXT,  
    Balance REAL,  
    IsVIP TEXT  
);
```

```
INSERT INTO Customers1 VALUES (1,'Praneeth',15000,'FALSE');  
INSERT INTO Customers1 VALUES (2,'Rahul',9000,'FALSE');  
INSERT INTO Customers1 VALUES (3,'Arjun',25000,'FALSE');  
INSERT INTO Customers1 VALUES (4,'Kiran',7000,'FALSE');
```

```
UPDATE Customers1  
SET IsVIP = 'TRUE'  
WHERE Balance > 10000;
```

```
SELECT * FROM Customers1;
```

Output Screenshot:

Scenario 2:

The screenshot displays a SQL IDE interface with the following components:

- Database Structure** | **Browse Data** | **Edit Pragmas** | **Execute SQL**
- SQL 1* (active tab)
- SQL Script:**

```
1 CREATE TABLE Customers1 (  
2     CustomerID INTEGER PRIMARY KEY,  
3     CustomerName TEXT,  
4     Balance REAL,  
5     IsVIP TEXT  
6 );  
7  
8 INSERT INTO Customers1 VALUES (1, 'Praneeth', 15000, 'FALSE');  
9 INSERT INTO Customers1 VALUES (2, 'Rahul', 9000, 'FALSE');  
10 INSERT INTO Customers1 VALUES (3, 'Arjun', 25000, 'FALSE');  
11 INSERT INTO Customers1 VALUES (4, 'Kiran', 7000, 'FALSE');  
12  
13 UPDATE Customers1  
14 SET IsVIP = 'TRUE'  
15 WHERE Balance > 10000;  
16  
17 SELECT * FROM Customers1;
```
- Result Table:**

	CustomerID	CustomerName	Balance	IsVIP
1	1	Praneeth	15000.0	TRUE
2	2	Rahul	9000.0	FALSE
3	3	Arjun	25000.0	TRUE
4	4	Kiran	7000.0	FALSE
- Execution Log:**

Execution finished without errors.
Result: 4 rows returned in 7ms
At line 17:
SELECT * FROM Customers1;

Scenario 3:

```
CREATE TABLE Loans1 (
```

```
    LoanID INTEGER PRIMARY KEY,
```

```
    CustomerName TEXT,
```

```
    DueDate TEXT
```

```
);
```

```
INSERT INTO Loans1 VALUES (101,'Praneeth',date('now','+10 day'));
```

```
INSERT INTO Loans1 VALUES (102,'Rahul',date('now','+45 day'));
```

```
INSERT INTO Loans1 VALUES (103,'Arjun',date('now','+20 day'));
```

```
INSERT INTO Loans1 VALUES (104,'Kiran',date('now','+5 day'));
```

```
SELECT
```

```
    CustomerName,
```

```
    'Reminder: Your loan is due on ' || DueDate AS Reminder
```

```
FROM Loans1
```

```
WHERE DueDate BETWEEN date('now') AND date('now','+30 day');
```

Output Screenshot:

Scenario 3:

The screenshot displays a SQL IDE interface with the following components:

- Database Structure** | **Browse Data** | **Edit Pragmas** | **Execute SQL**
- SQL 1*** editor showing the following SQL script:

```
1
2 CREATE TABLE Loans1 (
3     LoanID INTEGER PRIMARY KEY,
4     CustomerName TEXT,
5     DueDate TEXT
6 );
7
8 INSERT INTO Loans1 VALUES (101, 'Praneeth', date('now', '+10 day'));
9 INSERT INTO Loans1 VALUES (102, 'Rahul', date('now', '+45 day'));
10 INSERT INTO Loans1 VALUES (103, 'Arjun', date('now', '+20 day'));
11 INSERT INTO Loans1 VALUES (104, 'Kiran', date('now', '+5 day'));
12
13 SELECT
14     CustomerName,
15     'Reminder: Your loan is due on ' || DueDate AS Reminder
16 FROM Loans1
17 WHERE DueDate BETWEEN date('now') AND date('now', '+30 day');
```
- Result Table:**

	CustomerName	Reminder
1	Praneeth	Reminder: Your loan is due on ...
2	Arjun	Reminder: Your loan is due on ...
3	Kiran	Reminder: Your loan is due on ...

Execution finished without errors.
Result: 3 rows returned in 5ms
At line 13:
SELECT
 CustomerName,
 'Reminder: Your loan is due on ' || DueDate AS Reminder
FROM Loans1
WHERE DueDate BETWEEN date('now') AND date('now', '+30 day');

Exercise 3: Stored Procedures:

Scenario 1:

```
CREATE TABLE Accounts (  
    AccountID INTEGER PRIMARY KEY,  
    CustomerName VARCHAR(50),  
    AccountType VARCHAR(20),  
    Balance INTEGER  
);
```

```
INSERT INTO Accounts VALUES (101, 'Praneeth', 'Savings', 50000);  
INSERT INTO Accounts VALUES (102, 'Rahul', 'Savings', 30000);  
INSERT INTO Accounts VALUES (103, 'Arjun', 'Current', 40000);  
INSERT INTO Accounts VALUES (104, 'Kiran', 'Savings', 70000);
```

```
UPDATE Accounts  
SET Balance = Balance * 1.01  
WHERE AccountType = 'Savings';
```

```
SELECT * FROM Accounts;
```

Output Screenshot:

Scenario 1:

The screenshot displays a SQL IDE interface with the following components:

- Database Structure** | **Browse Data** | **Edit Pragmas** | **Execute SQL**
- SQL 1*
- SQL Script:

```
2
3 CREATE TABLE Accounts (
4     AccountID INTEGER PRIMARY KEY,
5     CustomerName VARCHAR(50),
6     AccountType VARCHAR(20),
7     Balance INTEGER
8 );
9 INSERT INTO Accounts VALUES (101, 'Praneeth', 'Savings', 50000);
10 INSERT INTO Accounts VALUES (102, 'Rahul', 'Savings', 30000);
11 INSERT INTO Accounts VALUES (103, 'Arjun', 'Current', 40000);
12 INSERT INTO Accounts VALUES (104, 'Kiran', 'Savings', 70000);
13
14 UPDATE Accounts
15 SET Balance = Balance * 1.01
16 WHERE AccountType = 'Savings';
17
18 SELECT * FROM Accounts;
19
20 //-----
21 //scenario 2:
```
- Execution Results:**

	AccountID	CustomerName	Balance
1	201	Praneeth	45000.0
2	202	Rahul	30000.0
- Execution Log:**

```
Execution finished without errors.
Result: 2 rows returned in 6ms
At line 18:
SELECT * FROM Accounts;
```

Scenario 2:

```
CREATE TABLE Employees (  
    EmployeeID INTEGER PRIMARY KEY,  
    EmployeeName TEXT,  
    Department TEXT,  
    Salary REAL  
);
```

```
INSERT INTO Employees VALUES (1,'John','IT',50000);  
INSERT INTO Employees VALUES (2,'David','HR',45000);  
INSERT INTO Employees VALUES (3,'Alice','IT',60000);  
INSERT INTO Employees VALUES (4,'Bob','Finance',55000);
```

```
UPDATE Employees  
SET Salary = Salary + (Salary * 10 / 100)  
WHERE Department = 'IT';
```

```
SELECT * FROM Employees;
```

Output Screenshot:

Scenario 2:

Database Structure | Browse Data | Edit Pragmas | Execute SQL

SQL 1*

```
21 //scenario 2:
22
23 CREATE TABLE Employees (
24     EmployeeID INTEGER PRIMARY KEY,
25     EmployeeName TEXT,
26     Department TEXT,
27     Salary REAL
28 );
29
30 INSERT INTO Employees VALUES (1, 'John', 'IT', 50000);
31 INSERT INTO Employees VALUES (2, 'David', 'HR', 45000);
32 INSERT INTO Employees VALUES (3, 'Alice', 'IT', 60000);
33 INSERT INTO Employees VALUES (4, 'Bob', 'Finance', 55000);
34
35 UPDATE Employees
36 SET Salary = Salary + (Salary * 10 / 100)
37 WHERE Department = 'IT';
38
39 SELECT * FROM Employees;
40
```

	EmployeeID	EmployeeName	Department	Salary
1	1	John	IT	55000.0
2	2	David	HR	45000.0
3	3	Alice	IT	66000.0
4	4	Bob	Finance	55000.0

Execution finished without errors.
Result: 4 rows returned in 6ms
At line 39:
SELECT * FROM Employees;

Scenario 3:

DROP TABLE IF EXISTS Accounts;

```
CREATE TABLE Accounts (  
    AccountID INTEGER PRIMARY KEY,  
    CustomerName TEXT,  
    Balance REAL  
);
```

INSERT INTO Accounts VALUES (201,'Praneeth',50000);

INSERT INTO Accounts VALUES (202,'Rahul',25000);

```
UPDATE Accounts  
SET Balance = Balance - 5000  
WHERE AccountID = 201  
AND Balance >= 5000;
```

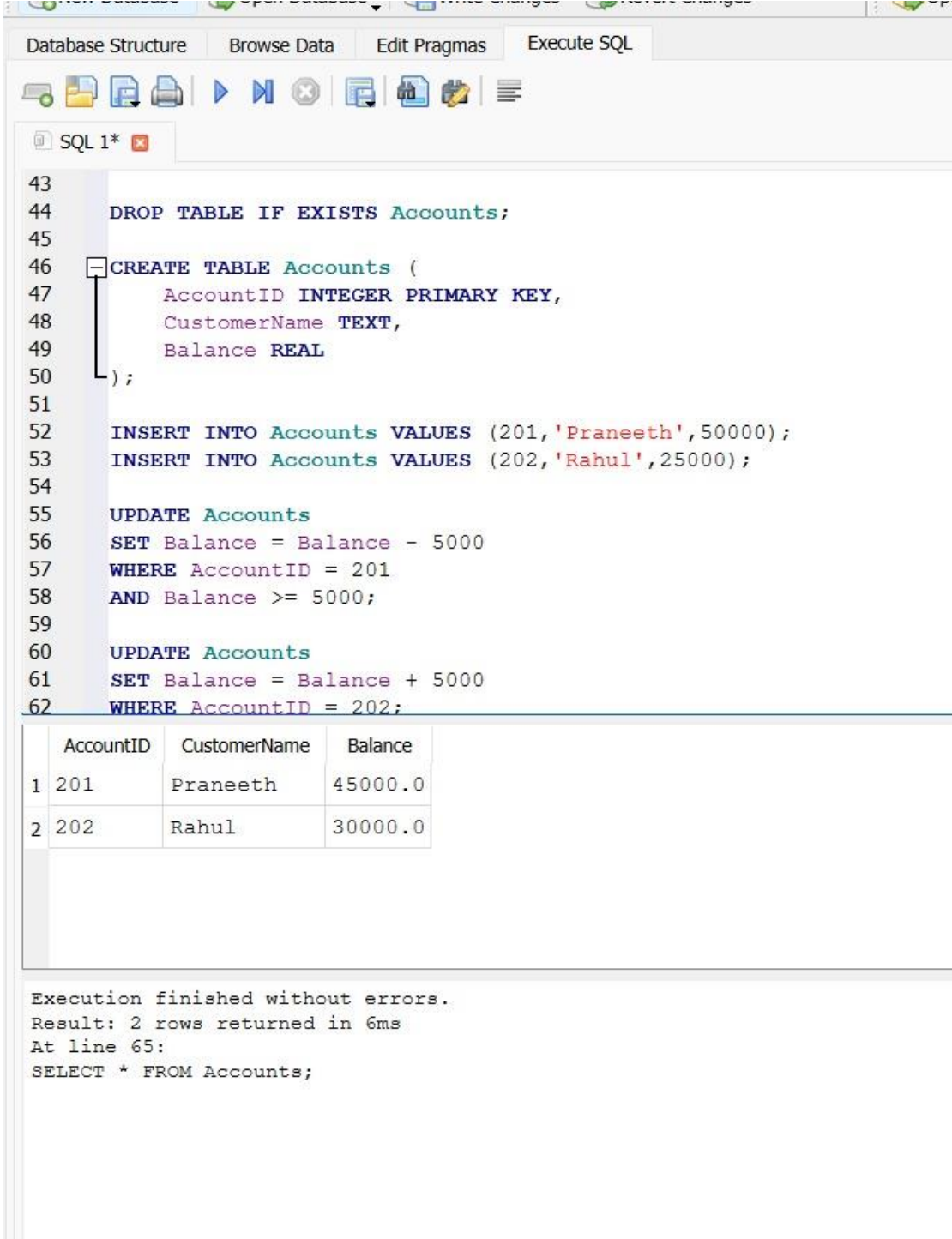
```
UPDATE Accounts  
SET Balance = Balance + 5000  
WHERE AccountID = 202;
```

COMMIT;

SELECT * FROM Accounts;

Output Screenshot:

Scenario 3:



The screenshot displays a SQL IDE interface with a script editor and a results pane. The script editor shows a series of SQL commands for creating a table, inserting data, and updating balances. The results pane shows the output of the final SELECT statement, which is a table with 2 rows and 3 columns: AccountID, CustomerName, and Balance.

```
43
44 DROP TABLE IF EXISTS Accounts;
45
46 CREATE TABLE Accounts (
47     AccountID INTEGER PRIMARY KEY,
48     CustomerName TEXT,
49     Balance REAL
50 );
51
52 INSERT INTO Accounts VALUES (201, 'Praneeth', 50000);
53 INSERT INTO Accounts VALUES (202, 'Rahul', 25000);
54
55 UPDATE Accounts
56 SET Balance = Balance - 5000
57 WHERE AccountID = 201
58 AND Balance >= 5000;
59
60 UPDATE Accounts
61 SET Balance = Balance + 5000
62 WHERE AccountID = 202;
```

	AccountID	CustomerName	Balance
1	201	Praneeth	45000.0
2	202	Rahul	30000.0

Execution finished without errors.
Result: 2 rows returned in 6ms
At line 65:
SELECT * FROM Accounts;